COURSE : STIQK2113
DISCRETE STRUCTURE

6.1 TREES
6.2 LABELED TREE
6.3 TREE SEARCHING

Dr. Munya Saleh Saeed Ba Matraf

6.1 TREES

Trees

- Let A be a set and let T be a relation on A.
- We say that T is a tree

  if there is a vertex v_o in A with the property that there exists a unique path in T from v_o to every other vertex in A, but no path from v_o to v_o.

Trees

- v_o is called root of the tree T and T is then referred to as a rooted tree.
- If (T, v_o) is a rooted tree on the set A, an element of v of A will often be referred to as a vertex in T.

Trees

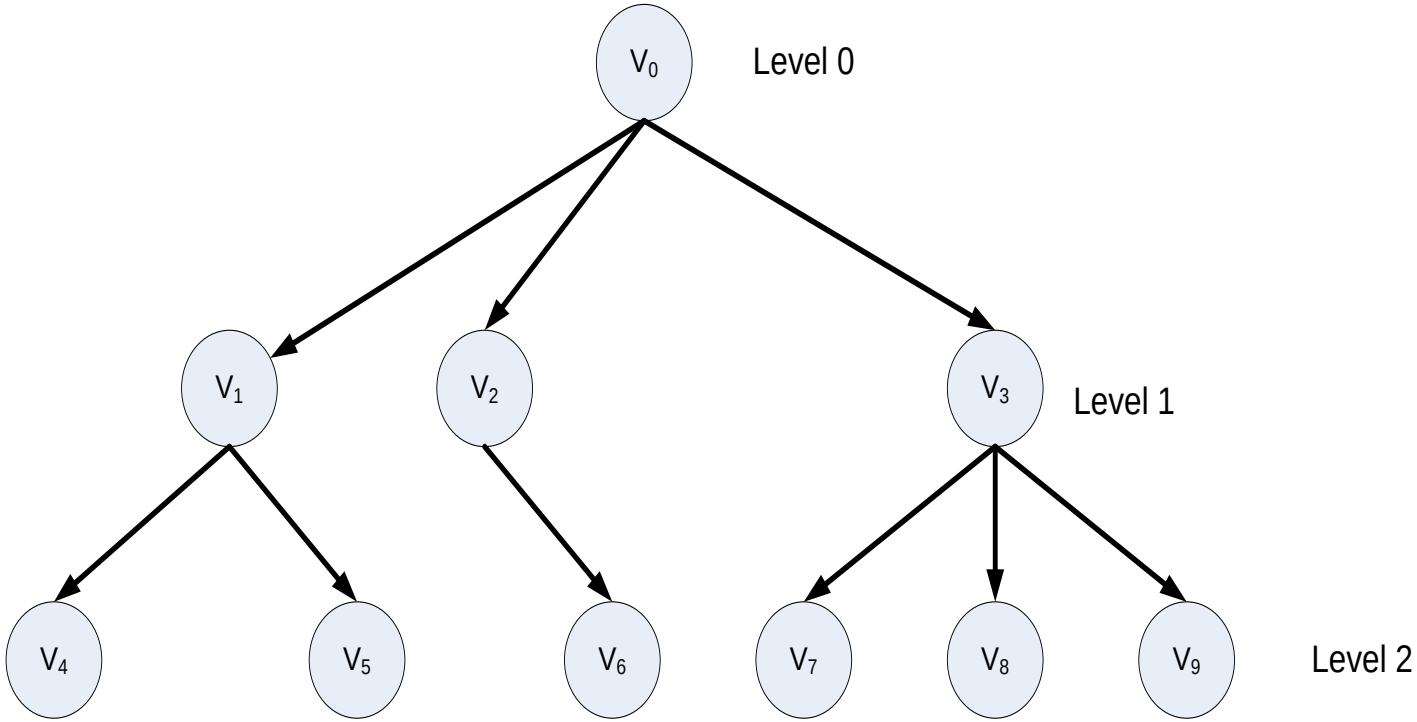
- Theorem 1

  Let (T, v_o) be a rooted tree. Then

  - There are no cycles in T
  - v_o is the only root of T
  - Each vertex in T, other then v_o has in-degree one, and v_o has in-degree zero.

Trees

- The diagraph of tree from Theorem 1 can be describe as below



Trees

- Explanations from the diagraph
  - v_o is a parent of these level 1 vertices
  - The level 1 vertices are called offspring of v_o
  - Example:
    - v_3 called the parent of the three offspring v_7,v_8 and v_9
    - The offspring of any one vertex are sometimes called siblings.
    - The largest level number of a tree is called the height of the tree.
    - The vertices of the tree that have no offspring are called the leaves of the tree.
    - The offspring for each vertex is ordered tree. If a vertex have four offspring, we may assume that they are ordered, so may refer to them as the first, second, third or fourth.

Trees

- Theorem 2

  Let (T, v_o) be a rooted tree on a set A. Then

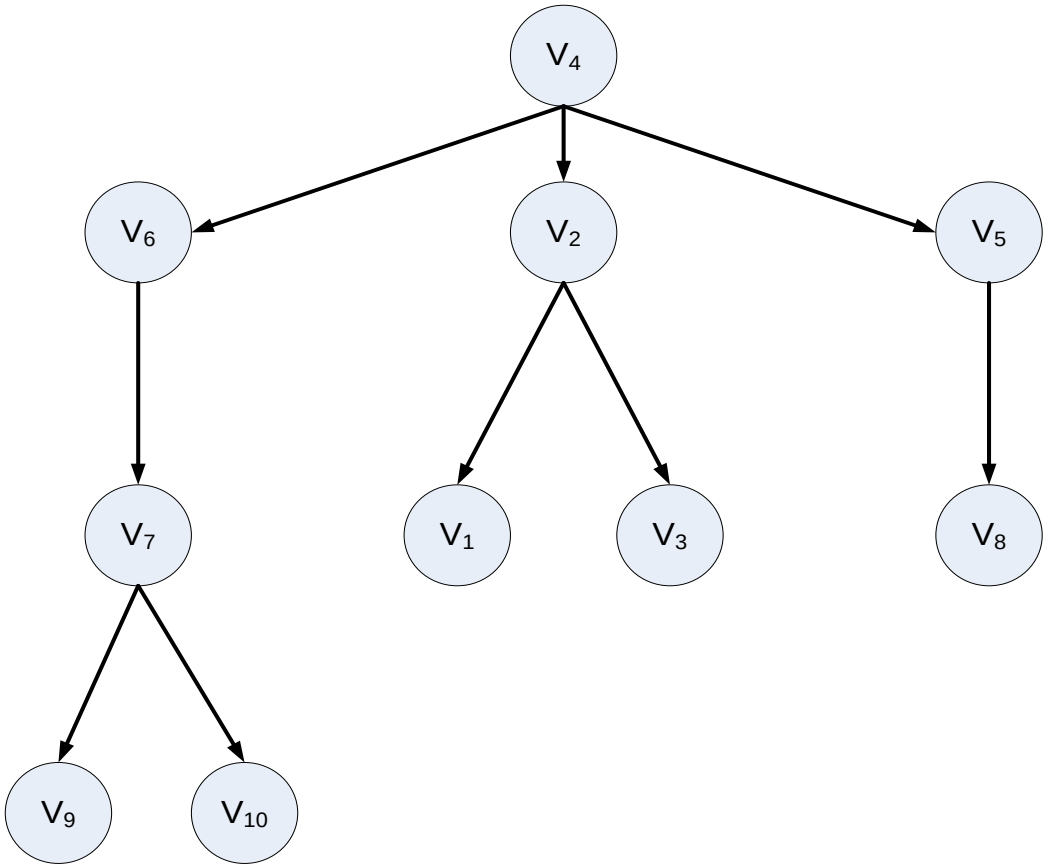
  - T is irreflexive
  - T is asymmetric
  - If (a,b) $\in$ T and (b,c) $\in$ T, then (a,c) $\notin$ T, for all a, b, and c in A.

Trees

- **Example 1**
  - Let A = $\{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8, v_9, v_{10}\}$ and let T = $\{(v_2, v_3), (v_2, v_1), (v_4, v_5), (v_4, v_6), (v_5, v_8), (v_6, v_7), (v_4, v_2), (v_7, v_9), (v_7, v_{10})\}$.

  Show that T is a rooted tree and identify the root

Trees



Trees

- From Example 1
  - If n is a positive integer, a tree T is an n-tree if every vertex has at most n offspring.
  - If all vertices of T, other than the leaves, have exactly n offspring, T is an complete n-tree.
  - A 2-tree is often called a binary tree.
  - A complete 2-tree is often called a complete binary tree.
  - Descendants refer to all vertices of T that can be reached by a path beginning at v.

Trees

- Theorem 3

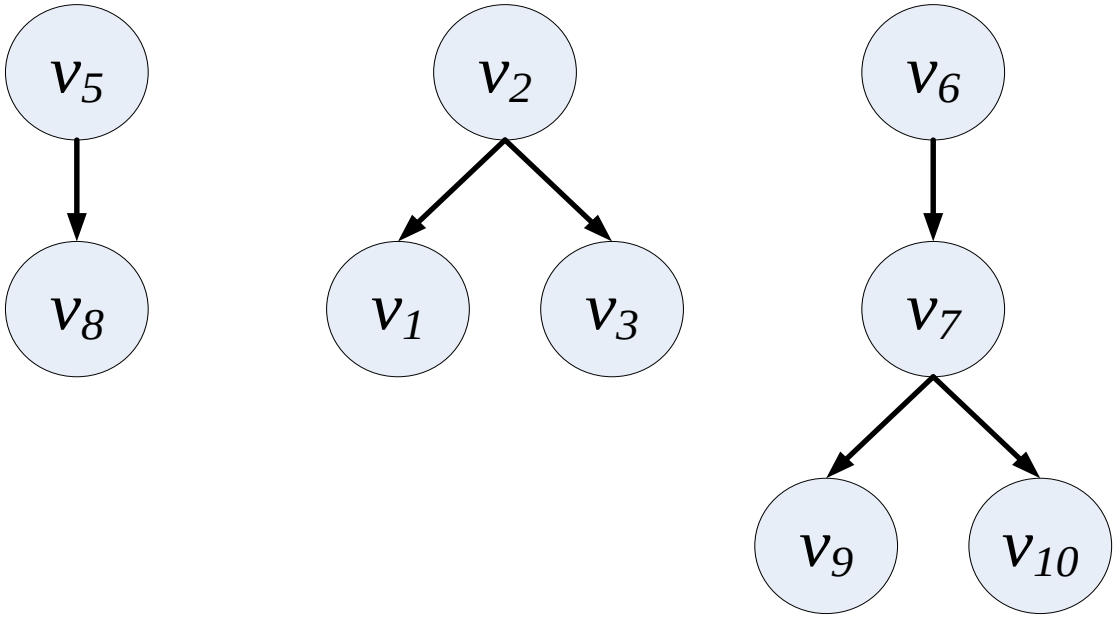
  If (T, v_o) be a rooted tree and v ∈ T, then T(v) is also a rooted tree with root v. T(v) is the subtree of T beginning at v.

Trees

- Example
  - Consider the tree of example 1. This tree has root v_4 and subtree T(v_5), T(v_2) and T(v_6) of T.

Trees

- Answer



Trees

- Exercise
  - In exercise 1-3, each relation R is define on the set A. In each case determine if R is a tree and if it is, find the root.

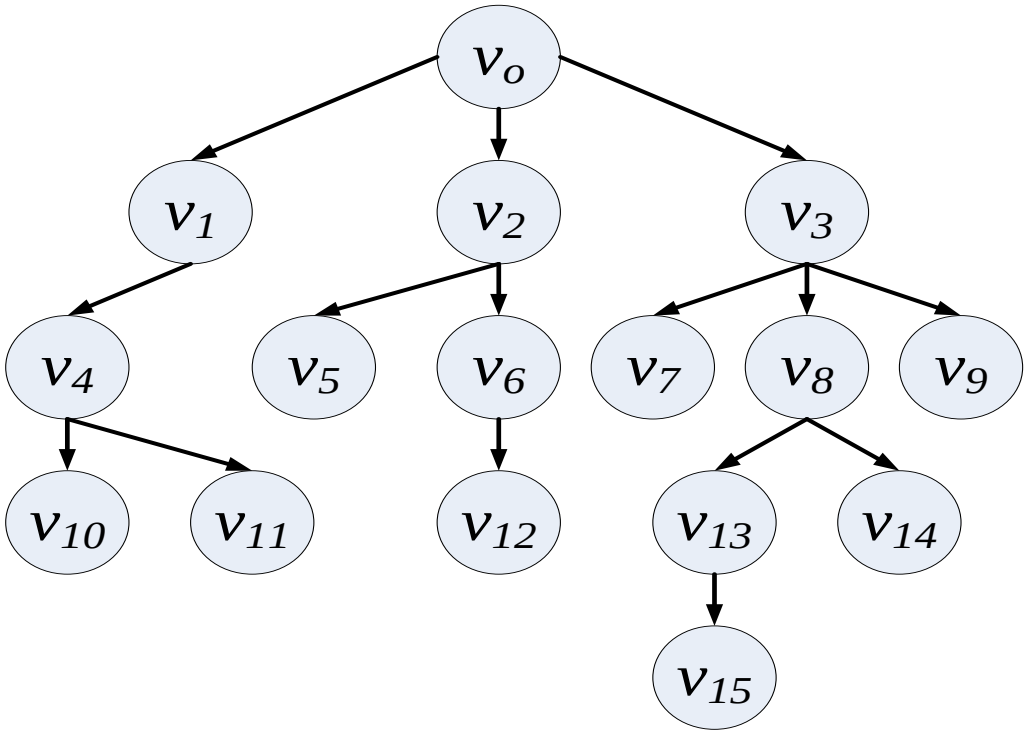
1. A = {a, b, c, d, e}, R = {(a, d), (b, c), (c, a), (d, e)}.

2. A = {1, 2, 3, 4, 5, 6}, R = {(1, 1), (2, 1), (2, 3), (3,4), (4, 5), (4, 6)}

3. A = {t, u, v, w, x, y, z}, R = {(t, u), (u, w), (u, x), (u, v), (v, z), (v, y)}

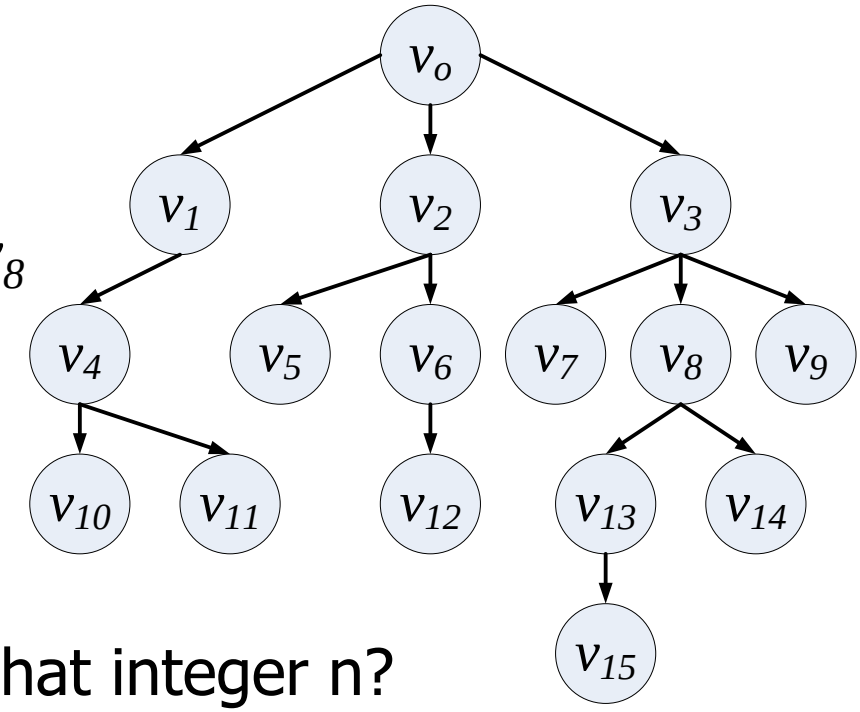
Trees

- Exercises
  - Consider the rooted tree (T, v_o) below



Trees

a) List all level 3 vertices
b) List all leaves
c) What are the sibling of v_8
d) What are the descendants of v_8
e) Compute the tree T(v_2)
f) Compute the tree T(v_3)
g) What is the height of (T, v_o)
h) What is the height of (T, v_3)
i) Is (T, v_o) an n-tree? If so, for what integer n?
j) Is T(, v_o) a complete n-tree? If so, for what integer n?



6.2 LABELED TREE

Labeled Tree

- Label the vertices or edges of a diagraph to indicate that the diagraph is being used for particular purpose.
- The vertices represent as dot and show the label of each vertex next to the dot representing that vertex.
- The labeled trees known as ordered tree

Labeled Tree

- **Example 1**

  Consider parenthesized, algebraic expression below.

$$(3 - (2 * x)) + ((x - 2) - (3 + x))$$

Labeled Tree

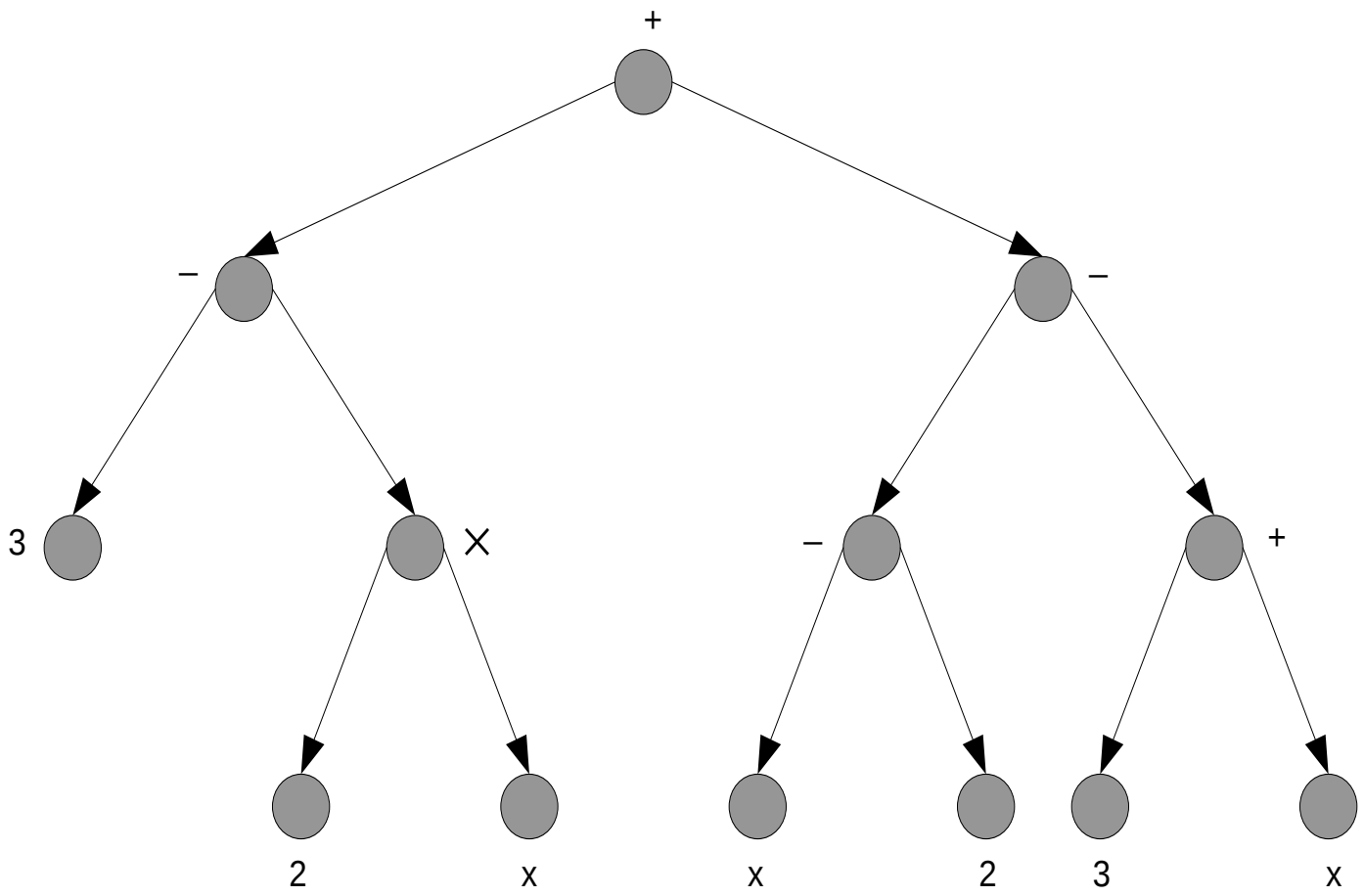
The graphical representation for expression above known as labeled binary tree.

In this tree the root is labeled with the central operator of the main expression

The offspring of the root are labeled with the central operator of the expression for the left and right arguments respectively.

Labeled Tree

(3 - (2 * x)) + ((x – 2) – (3 + x))



Labeled Tree

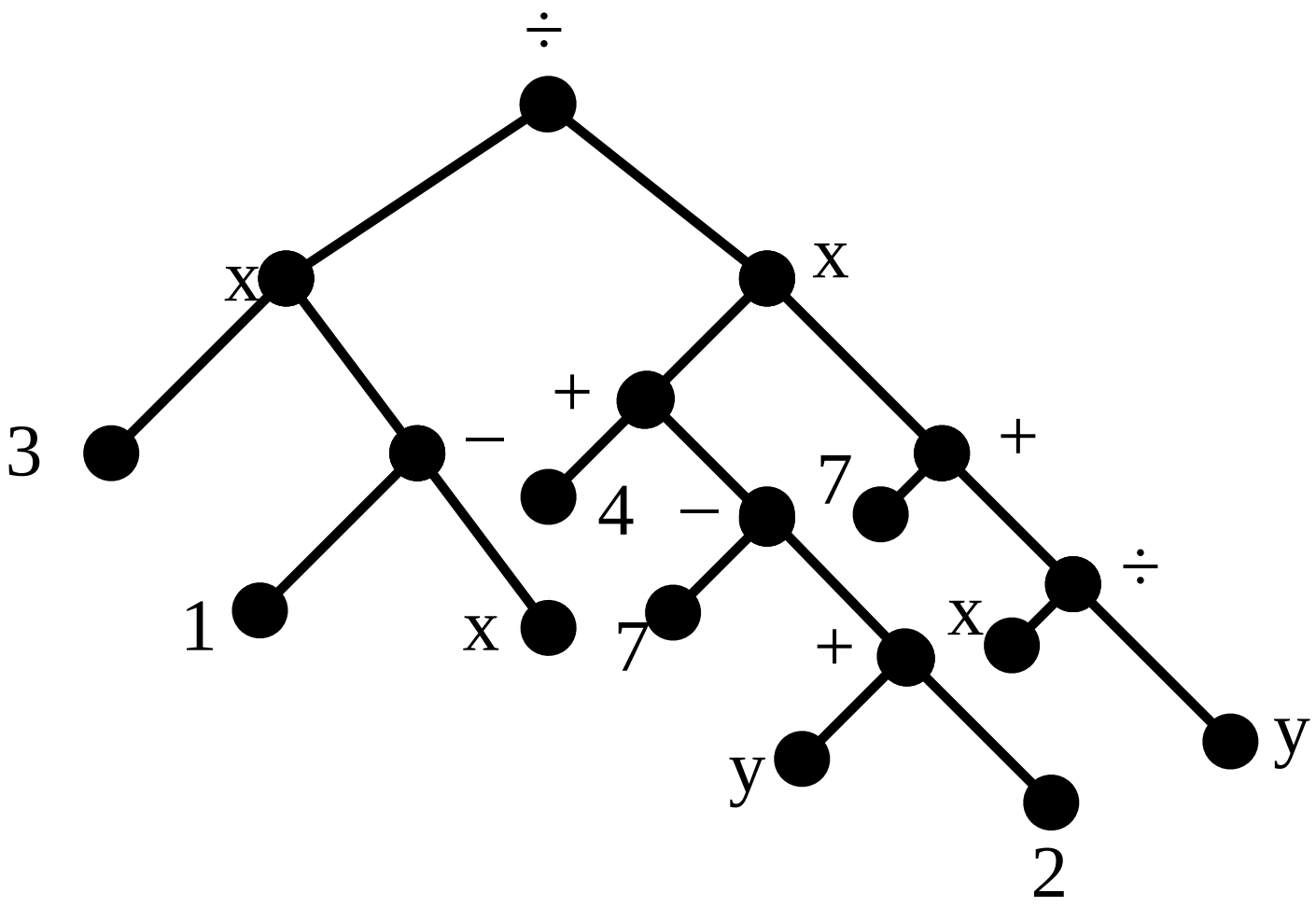
- **Example 2**

  Consider parenthesized, algebraic expression below.

$$((3\times(1-x))\div((4+(7-(y+2)))\times(7+(x\div y))))$$

Labeled Tree

$$((3\times(1-x))\div((4+(7-(y+2)))\times(7+(x\div y))))$$



- **Exercise**

$$(7+(6-2))-(x-(y-4))$$

$$(x+(y-(x+y)))\times((3\div(2\times7))\times4)$$

$$(11-(11\times(11+11)))+(11+(11\times11))$$

$$(x\div y)\div((x\times3)-(z\div4))$$

Labeled Tree

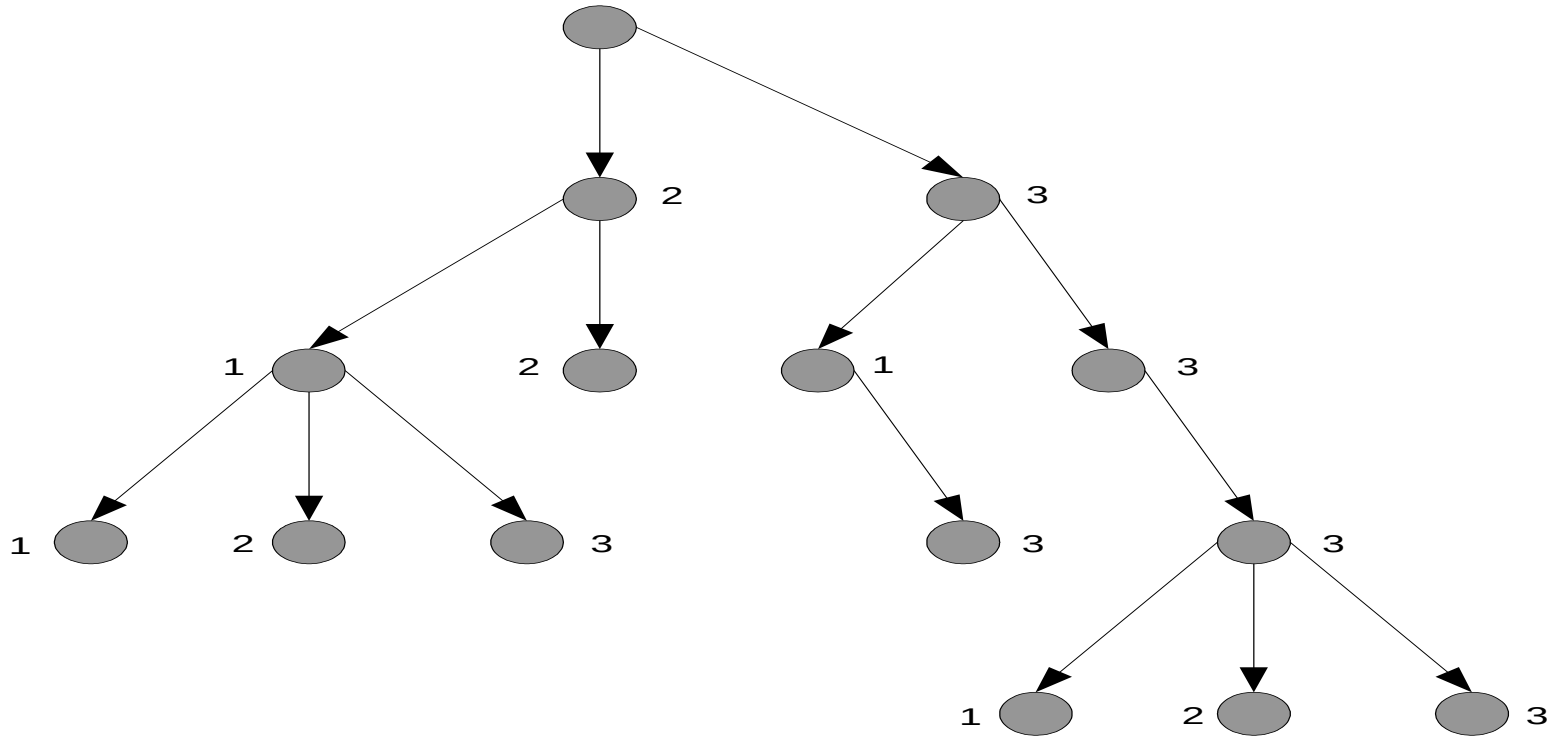
- **Example 3**

  Labeled tree is important for the computer implementation of a tree data structure.

  - Consider n-tree (T, v_o) given. Each vertex in T has at most n offspring.
  - Each vertex potentially has exactly n offspring, which would be ordered from 1 to n, but that some of the offspring in the sequence may be missing.
    - This labeled diagraph is called positional.

Labeled Tree

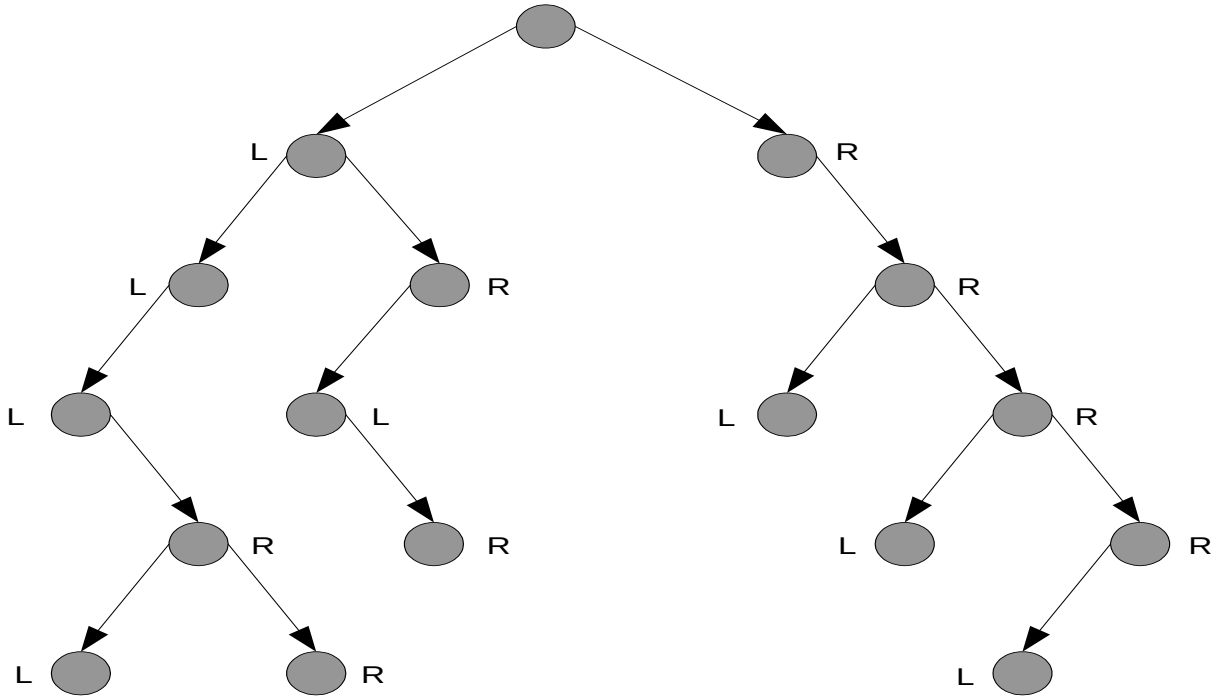
- The figure below shown the diagraph of a positional three-tree.



Labeled Tree

- **Example 4**

  For positional binary tree is shown below. Each offspring are often labeled left and right instead of 1 and 2.



6.3 TREE SEARCHING

Tree Searching

- A process of visiting each vertex in tree at some specific order
- There three type of tree searching
  - Preorder (TLR): Visit the root, then visit the left subtree, and finally the right subtree.
  - Inorder (LTR): Visit the left subtree first, then the root, and finally the right subtree
  - Postorder (LRT): Visit the left subtree, then the right subtree, and finally the root.

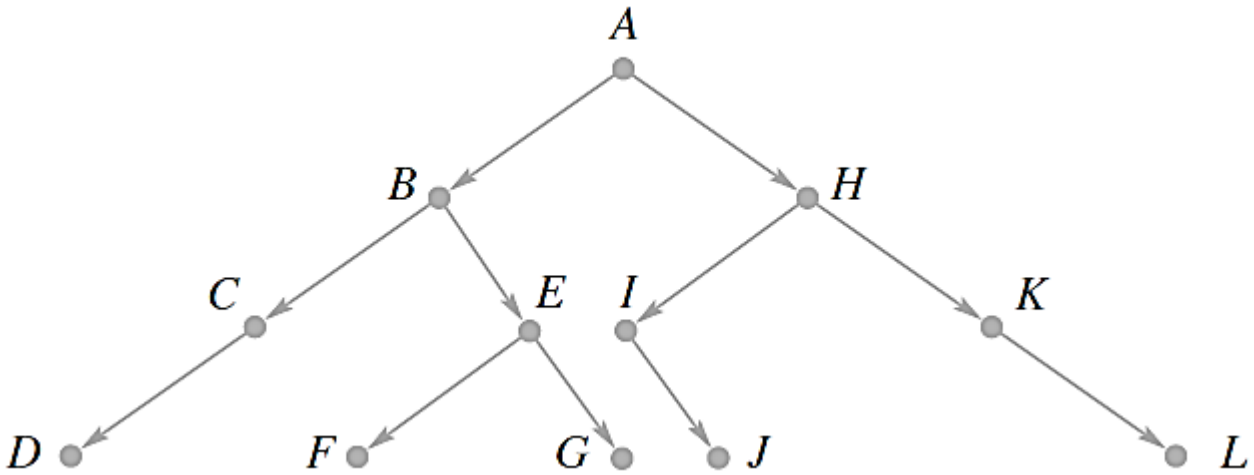
Tree searching (Preorder)

- Preorder Algorithm
  1. Visit v
  2. If v_L exist, call preorder algorithm (T(v_L), v_L).
  3. If v_R exist, call preorder algorithm (T(v_R), v_R).
  4. End algorithm
- Informally, the preorder search of a tree contains three steps
  1. Visit the root
  2. Search the left subtree if it exist
  3. Search the right subtree if it exist

  -> root->left->right

Example 1

- Let T be the labeled, positional binary tree whose digraph is shown below The root of this tree is the vertex labeled A. Suppose that, for any vertex v of T, visiting v prints out the label of v. Let us now apply the preorder search algorithm to this tree. Note first that if a tree consists only of one vertex, its root, then a search of this tree simply prints out the label of the root.

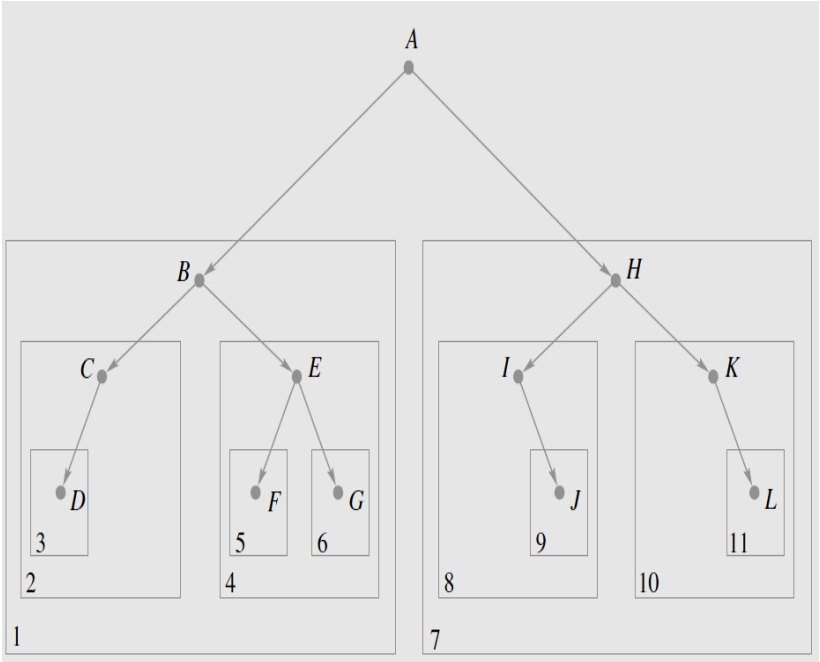


Answer

preorder traversal applied to tree **T**:

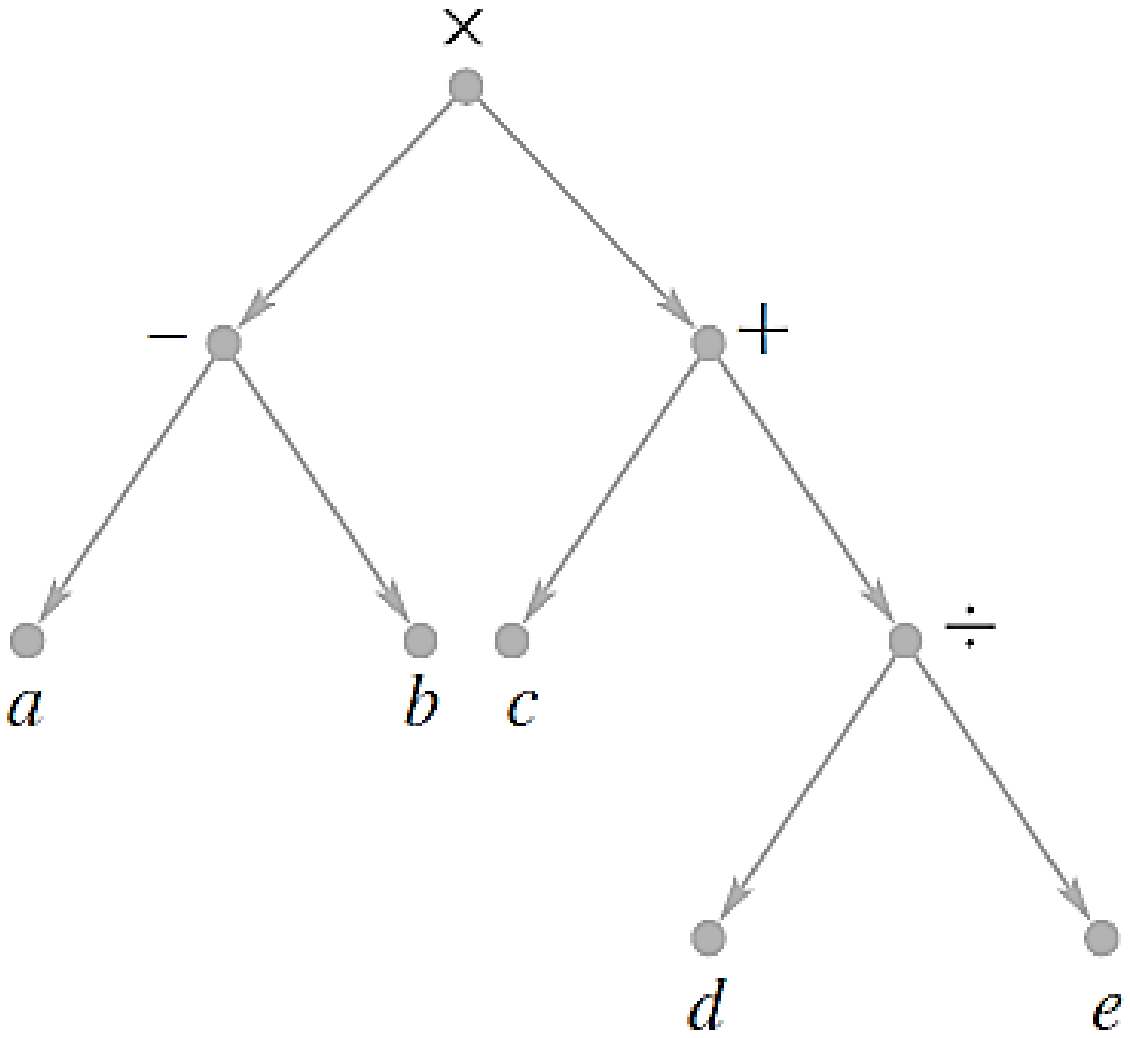
1. Visit and print the **root** (A).
2. **Search subtree 1**:
    1. Visit and print **B**.
    2. **Search subtree 2**:
        1. Print **C**.
        2. **Search subtree 3** (leaf node): Print **D**.
    3. **Search subtree 4**:
        1. Print **E**.
        2. **Search subtree 5** and **subtree 6**, producing **F** and **G**.
3. **Search subtree 7**, applying the same procedure:
    1. Produces the sequence **HIJKL**.
4. The final traversal results in printing **ABCDEFHIJKL**.

We have placed boxes around the subtrees of T and numbered these subtrees (in the corner of the boxes) to shows the order in which the algorithm PREORDER is applied to subtrees



Exercise

Apply a preorder traversal a
to the tree



Tree Searching (inorder)

- Informally, the inorder search of a tree contains three steps
  1. Search the left subtree if it exist
  2. Visit the root
  3. Search the right subtree if it exist

  -> left->root->right

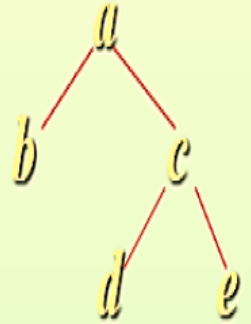
Tree Searching (postorder)

- Informally, the postorder search of a tree contains three steps
  1. Search the left subtree if it exist
  2. Search the right subtree if it exist
  3. Visit the root

-> left->right->root

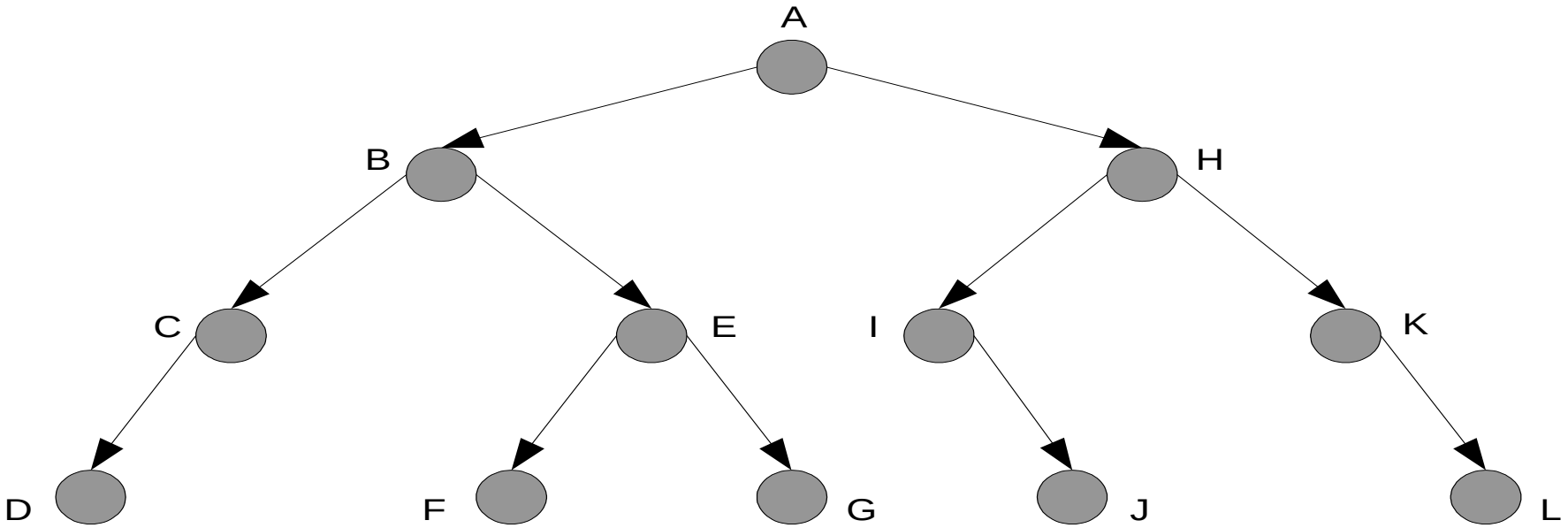
Summary

In-order (T)	Pre-order (T)	Post-order (T)
Algorithm ***Inorder(T)*** if $T \neq \emptyset$ $Inorder(T_{left})$ print(root of T) $Inorder(T_{right})$ Output: b a d c e	Algorithm ***Pre-order(T)*** if $T \neq \emptyset$ print(root of T) $Inorder(T_{left})$ $Inorder(T_{right})$ Output: a b c d e	Algorithm ***Post-order(T)*** if $T \neq \emptyset$ $Inorder(T_{left})$ $Inorder(T_{right})$ print(root of T) Output: b d e c a



Tree Searching

- Consider T is a positional binary tree with the diagraph is shown below. Get the result of T tree searching using inorder and postorder method.

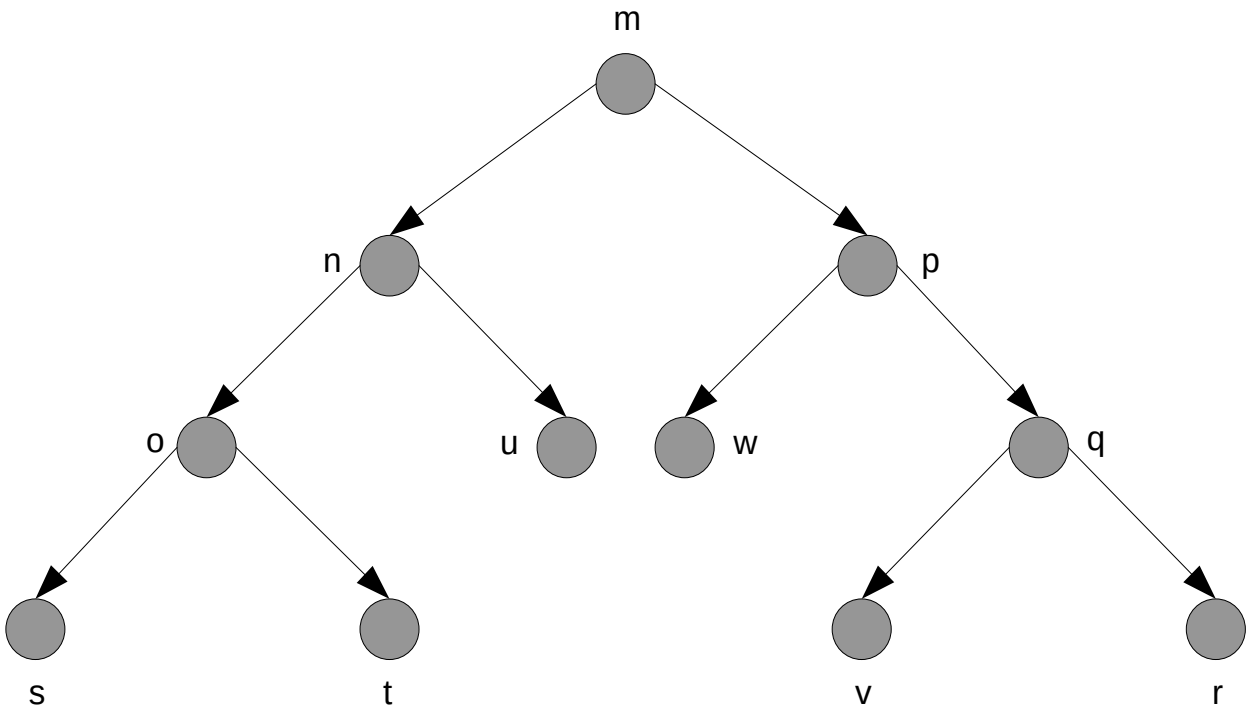


DCBFEGAIJHKL (in or post)
DCFGEBJILKHA (in or post)

Tree Searching

- **Exercise (HW)**

  Show the results of performing preorder, inorder and postorder search whose diagraph is shown below.



Tree Searching

- **Exercise**

  Show the results of performing preorder, inorder and postorder search whose expression is shown below.

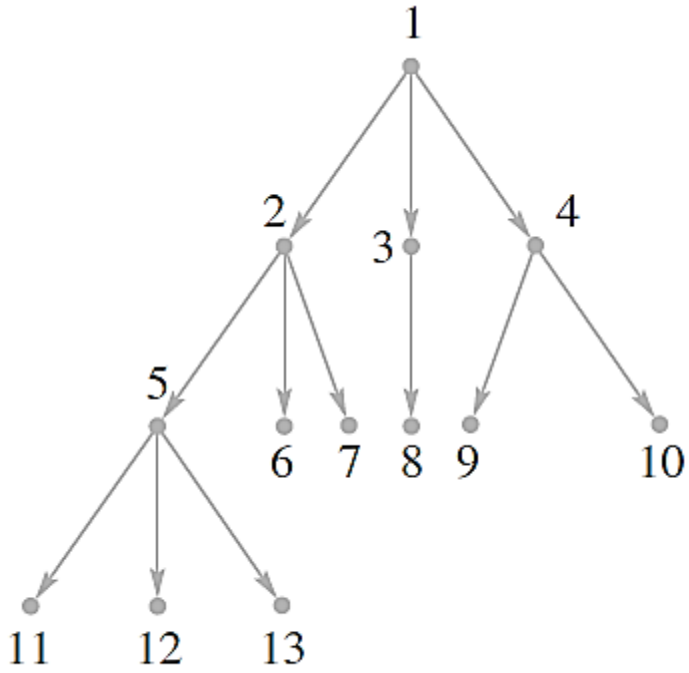
$$(x \div y) \div ((x \times 3) - (z \div 4))$$

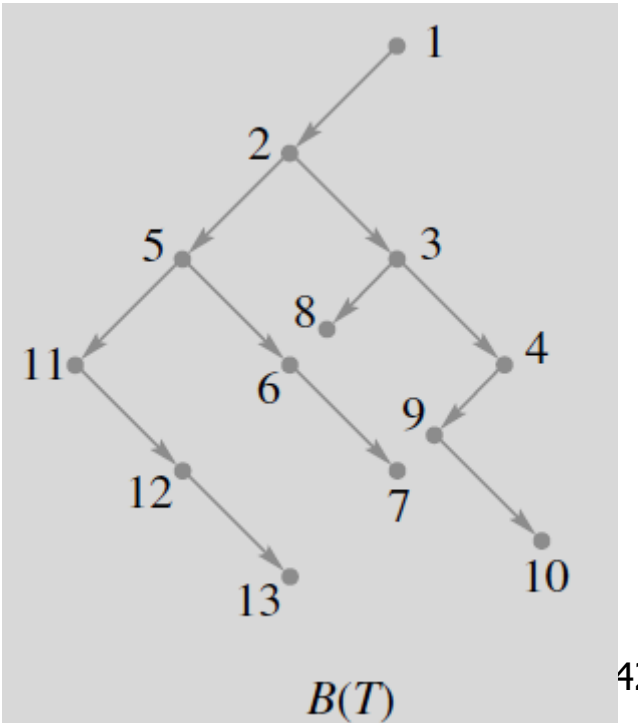
Searching General Trees

- Until now, we have only shown how to search binary trees.
- We now show that any ordered tree T may be represented as a binary tree and searching can be performed on them.
- Steps :
  - ordered tree defined as T
  - binary tree defined as B(T)
  - v as vertex in T and B(T)
  - left offspring of v in B(T) is the first offspring in T
  - right offspring of v in B(T) is the next sibling of v in T

Example

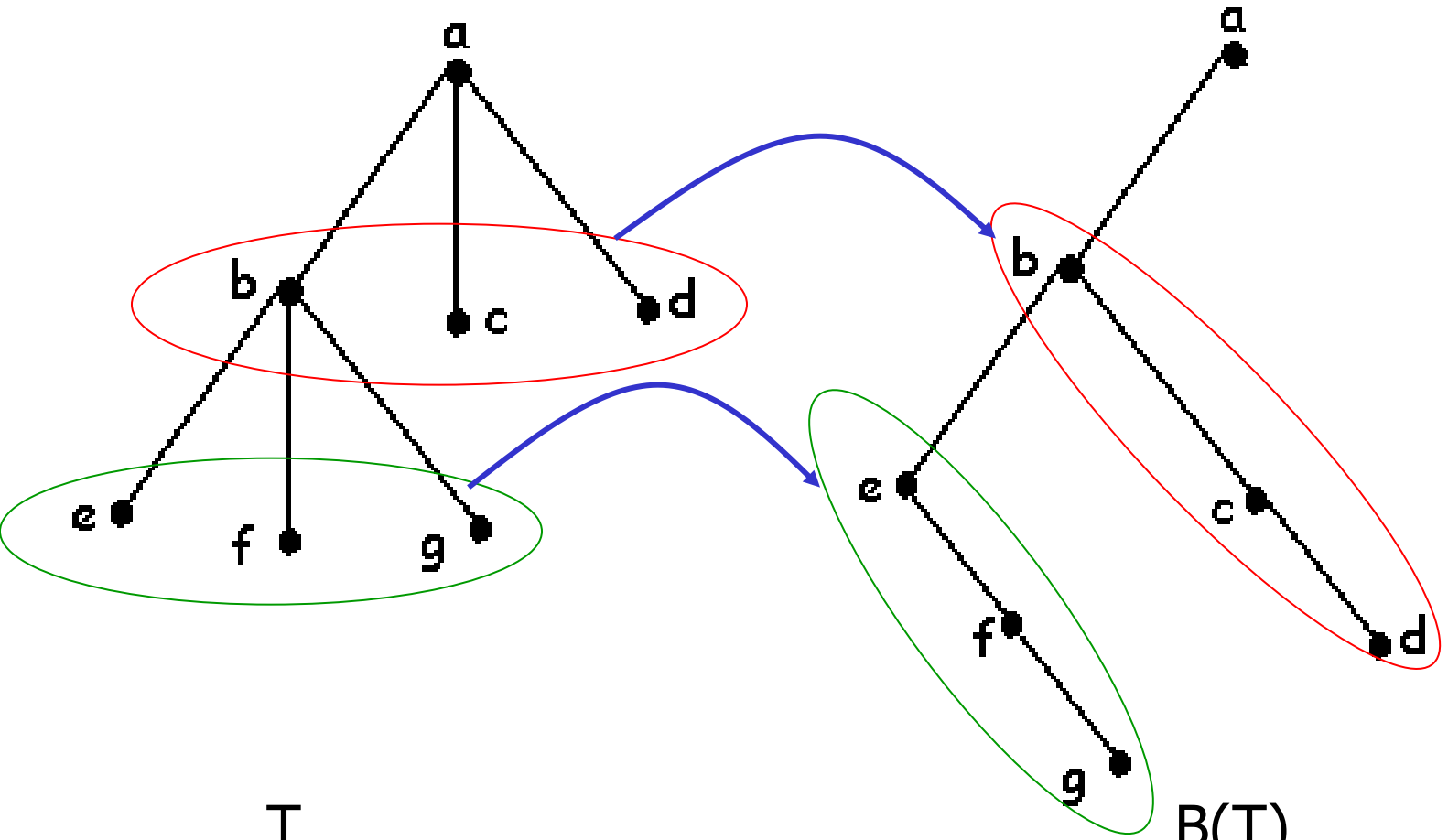
- simply draw a left edge from each vertex v to its first offspring (if v has offspring).
- Then draw a right edge from each vertex v to its next sibling (in the order given), if v has a next sibling.
- Thus the left edge from vertex 2, goes to vertex 5, because vertex 5 is the first offspring of vertex 2 in the tree T .
- Also, the right edge from vertex 2, goes to vertex 3, since vertex 3 is the next sibling in line (among all offspring of vertex 1).





$B(T)$

Example



T

B(T)

In Exercises 31 and 32 (Figures 24 and 25), draw the digraph of the binary positional tree $B(T)$ that corresponds to the tree shown. Label the vertices of $B(T)$ to show their correspondence to the vertices of T.

31.

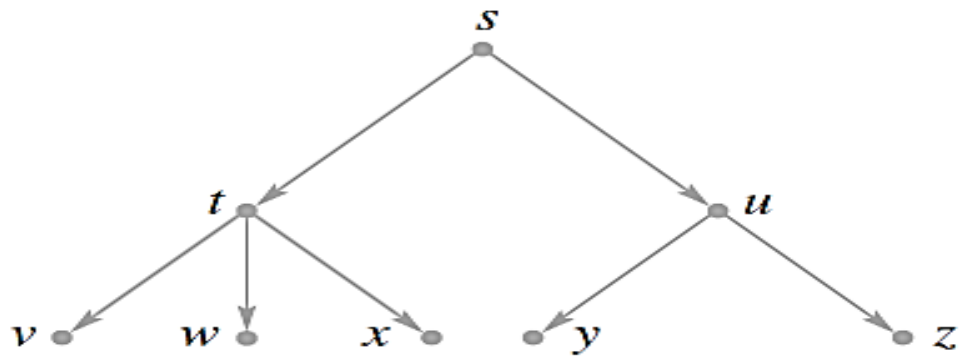


Figure 24

32.

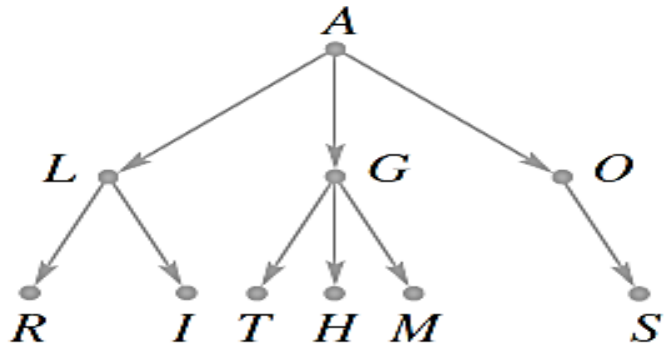


Figure 25